Exercises

Exercise 5: Exploratory analysis and graphics

Read Chapter 4 to help you complete the questions in this exercise.

1. As in previous exercises, either create a new R script or continue with your previous R script in your RStudio Project. Again, make sure you include any metadata you feel is appropriate (title, description of task, date of creation etc) and don't forget to comment out your metadata with a `#` at the beginning of the line.

2. You do not need to download anything for this exercise. You already have *'cardiacdata.txt'* in the `data` directory of your RStudio Project, from Exercise 3. If for some reason you haven't, go back to the **Data** link, download *'cardiacdata.xlsx'* and save it as a tab delimited file called *'cardiacdata.txt'* as you did before.

3. A quick reminder of what these data are. They come from a cohort study of the risk factors for cardiovascular disease, in which 163 adults aged between 55 and 75 were examined and a range of measurements taken: age and sex, systolic and diastolic blood pressure (mmHg), body mass index (kg m^{-2}), total cholesterol, HDL cholesterol and triglyceride (all mmol l^{-1}), units of alcohol drunk in the previous week, and smoking status. Note that you are importing the original file again, not the cleaned version you exported at the end of Exercise 4. Starting from the raw data every time, and doing your cleaning in a script, is what makes an analysis reproducible.

4. Import the *'cardiacdata.txt'* file into R using the `read.table()` function and assign it to a variable named `cardiac`. Use the `str()` function to display the structure of the dataset. As you found in Exercise 3, the `sex` and `smoking` variables are coded as integers, but they are really categories. Create a new variable in the `cardiac` dataframe for each of them, recoded as a factor with meaningful labels (`sex` is 1 for Female and 2 for Male; `smoking` is 1 for Current, 2 for Ex and 3 for Never). Use the `str()` function again to check the coding of these new variables. Almost every plotting function in this exercise treats a factor differently from a number, so this step is not optional housekeeping - get it wrong and your boxplots will come out as nonsense.

5. How many patients are there in each combination of smoking status and sex (hint: remember the `table()` function?)? Don't forget to use the factor recoded versions of these variables. Is the split between the smoking categories similar for women and men? Are any of the combinations small enough to worry about if you wanted to compare them? Finally, run the table again with the argument `useNA = "ifany"` so that the patients with no recorded smoking status appear as a row of their own. How many are there, and is there anything they have in common?

6. The humble cleveland dotplot is a great way of identifying if you have potential outliers in continuous variables (see Section 4.2.4). Create dotplots (using the `dotchart()` function) for the following variables; `bmi`, `systolic`, `tchol` and `alcohol`. Do these variables contain any unusually large or small observations? Don't forget, if you prefer to create a single figure with all 4 plots you can always split your plotting device into 2 rows and 2 columns (see Section 4.4 of the book).

7. That lone point away on the right of the `bmi` dotplot is the value you met in Exercise 4, when you ran `summary()` and found a maximum of 514.60. A body mass index of 514 is not possible: it is 51.46 with the decimal point in the wrong place. Notice how much less work the dotplot made of finding it. In Exercise 4 you had to know to go looking, read down a column of numbers and spot that one of them was absurd. Here it announces itself the moment you draw the plot. That's what dotplots are for, and it's why they're worth drawing for every continuous variable before you do anything else with them.

Remember that you imported the raw file again in Q4, so none of the cleaning you did in Exercise 4 is in this dataframe. Redo all three of those fixes now. Use the `which()` function to identify which observation the impossible `bmi` is, confirm the value with the square bracket `[ ]` notation, then set that `bmi`, the single `triglyceride` of 0 and the two `hdlchol` values of 0 to `NA`, exactly as you did in Exercise 4. Redraw the `bmi` dotchart. Now look again at all four plots from Q6. Several points still sit well away from the rest. Which ones, and what should you do about them?

8. Histograms are the standard way of looking at the distribution of a continuous variable (see Section 4.2.2). Create histograms for `bmi` and `alcohol`, either separately or in one figure as you did in Q6. Put the two side by side and you'll see they look nothing like each other. How would you describe the difference?

One thing to be careful of is that a histogram can look quite different depending on the number of 'breaks' used. The `hist()` function chooses breaks for you, but they are only a suggestion. Redraw the histogram of `systolic` twice, once with wide bins and once with narrow ones, and compare them. Which one would you show somebody, and what does the difference tell you about how much you should trust the shape of a histogram?

9. Straight on from that, then. Your histogram has identified a problem: `alcohol` is badly skewed, and skew is a nuisance in a plot because most of the points end up squashed into a corner. Naming the problem is the easy half. The other half is doing something about it, and then checking whether what you did has actually worked.

`alcohol` is a count. It is the number of units drunk in a week, so it is made of whole numbers and a good many of them are zero. For count data the square root is the standard choice, and it copes with the zeros without complaint because `sqrt(0)` is simply 0. Notice that this is a different answer from the one you used in Exercise 4 Q6, where you took logs of `triglyceride`, and the reason is the data rather than fashion. A blood concentration is a continuous measurement and takes a log happily. A count is not and does not.

Create a new variable in the `cardiac` dataframe holding the square root of `alcohol`, plot its histogram next to the untransformed one, and then check the result the way you checked Exercise 4 Q6, by looking at what happens to the gap between the mean and the median. Has it fixed the problem?

10. Scatterplots are great for visualising relationships between two continuous variables (Section 4.2.1). Before you draw one, decide which variable belongs on which axis. By convention the explanatory variable goes on the x axis and the response goes on the y axis, so the choice is not cosmetic.

Start with `bmi` and `systolic` blood pressure. Which is the explanatory variable, and which the response? Plot them the right way round. Now do the same for `hdlchol` and `triglyceride`. Is the choice as easy this time?

11. When visualising differences in a continuous variable between levels of a factor (categorical variable) then a boxplot is your friend (avoid using bar plots - Google 'bar plots are evil' for more info). Create a boxplot to visualise the differences in HDL cholesterol at each level of smoking status (don't forget to use the recoded version of this variable you created in Q4). Include x and y axis labels in your plot. Make sure you understand the anatomy of a boxplot before moving on - please ask if you're not sure (also see Section 4.2.3 of the book).

Now draw exactly the same plot for total cholesterol instead. Same patients, same factor, one line of code different, and the two figures do not say remotely the same thing. Which of the two shows a difference between the groups? And if you were writing this up, which would you be tempted to leave out?

12. An alternative to the boxplot is the violin plot, which combines a boxplot with a kernel density plot and shows you the shape of the distribution rather than just its quartiles. You will first need to install the `vioplot` package from CRAN and make it available with `library(vioplot)`. The `vioplot()` function then works in much the same way as `boxplot()`. Draw the HDL comparison from Q11 again as a violin plot and compare the two.

Every plot so far has appeared in the RStudio plot pane, which is fine while you're exploring but no use at all when you need the figure in a document. Getting a plot out of R is a three step sandwich. You open a graphics device that is a file rather than a window, you draw the plot exactly as you have been doing, and then you close the device with `dev.off()`. Nothing appears on screen while you do it, and if you forget the `dev.off()` your file will be unreadable (see Section 4.5 of the book). Save your violin plot as a pdf in the `output` directory you created in Exercise 1, then find the file and open it outside RStudio to check it really worked. Exercise 6 takes this further, including how to get a figure into Word or PowerPoint at a resolution that doesn't embarrass you.

13. To explore the relationships between several continuous variables at once it's hard to beat a pairs plot. Create a pairs plot for the variables; `age`, `systolic`, `diastolic`, `tchol`, `hdlchol`, `triglyceride` and `bmi` (see Section 4.2.5 of the book for more details). If it looks a little cramped in RStudio then click on the 'zoom' button in the plot viewer to see a larger version.

One of the useful things about `pairs()` is that you can choose what goes into each panel. Modify your plot to add a smoother (a wiggly line) to the lower triangle, which makes the shape of each relationship much easier to see. The book and `?pairs` show you how. Then, just by looking, which pair of variables is most strongly related, and which variable is related to almost nothing?

14. (optional, and a stretch) `pairs()` will put whatever you like in the upper triangle, including the correlation coefficients themselves, but there is no ready made function for it so you have to write one. This is a step beyond anything else in the exercise, and writing your own functions is not covered until the optional programming exercise, so treat it as a look ahead rather than something you are expected to manage now. If you fancy a go, `?pairs` has an example you can adapt. Do the numbers agree with what you concluded by eye?

End of Exercise 5